

catchmentACS: Isochrone-Based Catchment-Area Profiles from American Community Survey Data with Margin-of-Error Propagation

JoonHo Lee

ABSTRACT. Site-level analyses in education, public health, and planning often require a profile of the population living within a fixed travel time of each site on a list, such as the poverty rate around a proposed preschool classroom or the share of households receiving nutrition assistance near a clinic. In the United States such profiles are constructed from American Community Survey (ACS) tract estimates, which are published together with margins of error that are large at the tract scale. Existing R workflows delineate a drive-time isochrone around each site and interpolate tract values onto it, but the interpolation step returns point estimates only, so the published margins of error are discarded at the point where they are most needed. We describe the `catchmentACS` package, which assembles drive-time catchment profiles with margins of error carried through every stage, from the tract inputs to the derived rates, and with per-row provenance recording how each number was produced. The paper sets out the estimation framework, explains the design of the package, and works through single-site and multi-site illustrations, all of which can be reproduced without a Census API key or a routing account. A simulation study shows that the intervals the package reports are conservative, and that the site comparisons they support are not fragile to the independence assumption inherited from the Census Bureau approximation formulas. In an illustrative ranking of twenty catchments by poverty rate, about one in five pairwise comparisons could not be distinguished at the 90 percent level, which is information that interpolated point estimates cannot provide.

Keywords: American Community Survey; margin of error; areal interpolation; isochrone; catchment area; R

1. Introduction

Site-level analyses in education, public health, and planning often require a summary of the population living within a fixed travel time of a school, a clinic, or another service site. A state preschool program that allocates funds by community need requires the poverty rate around each site, and a health department siting a clinic may require the number of households receiving nutrition assistance within a short drive. In the United States,

Date: August 2026

Lee: Department of Educational Studies in Psychology, Research Methodology, and Counseling, College of Education, The University of Alabama, Tuscaloosa, AL 35487, USA. jlee296@ua.edu. ORCID: 0009-0006-4019-8703.

the standard source for such small-area characteristics is the American Community Survey (ACS), whose five-year tract-level estimates cover the country on a rolling basis (Walker, 2023). The setting that motivated the package described in this paper is of this kind: Alabama’s state-funded preschool program weighs the economic need of the community that a proposed classroom would serve, and when enrollment records do not yet exist for a new site, characteristics of the surrounding community are the only measure available.

Because the ACS is a sample survey, every published estimate is accompanied by a margin of error (MOE), and at the tract scale these margins are large. Spielman et al. (2014) found tract-level margins of error to be on average 75 percent larger than those of the corresponding 2000 long-form census estimates, with the greatest uncertainty concentrated in the small, heterogeneous areas that site-level analysis examines, and Folch et al. (2016) showed that this uncertainty is itself spatially patterned, so that the estimates least suited to fine-grained comparison are not distributed at random across a state. The Census Bureau asks users to carry this uncertainty into derived quantities and provides approximation formulas for doing so (U.S. Census Bureau, 2020). In planning practice, however, margins of error are widely misunderstood and frequently left unreported (Jurjevich et al., 2018), and the problem is compounded when tract estimates are transferred onto site catchment areas, because the transfer step in common workflows returns point estimates only.

Uncertainty in small-area ACS estimates can be addressed in two broad ways. In the model-based approach, tract estimates are smoothed, pooled, or regionalized so that the published error is reduced before the estimates are used; the data-driven regionalization of Spielman and Folch (2015) is of this kind. In the design-based approach, which the package described here follows, the published margins of error are carried through every subsequent operation and reported alongside the results, so that the uncertainty remains visible to whoever reads them. The two approaches are complementary, and an analyst who requires stable estimates for very small areas may reasonably prefer the first. Our concern is with recurring policy pipelines that must report the Census Bureau’s own published uncertainty without alteration, because the resulting numbers enter an administrative process that is subject to audit.

A mature ecosystem of R packages covers the individual steps of the catchment-profiling workflow. `tidycensus` (Walker & Herman, 2026) and `tigris` (Walker, 2025) retrieve ACS estimates, margins of error, and tract geometries. Drive-time isochrones are available from several routing engines through `osrm` (Giraud, 2022), `mapboxapi` (Walker, 2026), and `r5r` (Pereira et al., 2021), as well as the `openrouteservice` package on GitHub. Interpolation of tract attributes onto overlapping polygons is provided by `st_interpolate_aw()` in `sf` (Pebesma, 2018), by `areal` (Prenner & Revord, 2019), and, with population weighting, by `interpolate_pw()` in `tidycensus`; Walker (2023) documents how these pieces are combined

into a catchment-profiling workflow. A related but distinct family of methods, represented on CRAN by [accessibility](#) (Pereira & Herszenhut, 2026), computes floating catchment area indices of service supply relative to demand (Luo & Wang, 2003). Since these methods score access to facilities rather than summarize the population around a site, we do not consider them further.

The closest prior art for the problem addressed here is the `acs` package (Glenn, 2019), which stored the published margins of error alongside the estimates and propagated them through arithmetic on `acs` objects by the same root-sum-of-squares rule that the Census Bureau recommends. That package was withdrawn from CRAN in 2019 and is available only from the archive. More importantly, its arithmetic combines whole geographic units, and it offers no facility for the fractional weights that arise when tract boundaries are cut by a catchment. Within the profiling workflow itself, every interpolation step returns point estimates. As far as we are aware, no maintained R package propagates ACS margins of error through catchment-area aggregation, so an analyst who requires uncertainty for a catchment profile must derive the weighted error formulas and implement them by hand.

In this paper we describe [catchmentACS](#) (Lee, 2026), an R package that turns a table of site coordinates into drive-time catchment profiles of ACS characteristics with margins of error propagated from tract inputs to derived rates. A single orchestrated call generates or accepts isochrones, retrieves or accepts tract estimates, aggregates them with area weights, combines margins of error by weighted extensions of the Census Bureau approximation formulas, derives rates with ratio margins of error, and attaches per-row provenance recording how each number was produced. Since the package is intended for recurring policy pipelines, particular attention has been paid to deterministic replay, input validation, per-site failure isolation, caching, and offline reproducibility from bundled data. The package is available on CRAN, with development hosted at <https://github.com/joonho112/catchmentACS> and documentation at <https://joonho112.github.io/catchmentACS/>.

The plan of the paper is as follows. In Section 2 we set out the estimation framework, namely area-weighted aggregation over tract intersections and the margin-of-error propagation rules attached to it. Section 3 describes the package and the design decisions behind its interface. In Section 4 we work through a single catchment in Birmingham, Alabama. Section 5 examines how well the propagated margins of error describe the sampling variability they summarize, and what the assumptions of Section 2 cost. In Section 6 we profile a batch of twenty sites and examine what the margins of error imply for comparisons between sites. Section 7 compares the package with general-purpose areal interpolation tools. Finally, we close the paper with a discussion in Section 8. Appendices A to E form an online supplement containing the variable catalog, the output contract, the batch workload, and the full simulation evidence behind Section 5.

2. Area-weighted catchment estimation with ACS data

Consider a set of sites, each with point coordinates, and one or more travel-time thresholds (for instance, 5, 10, and 15 minutes of driving). Around each site a routing engine produces an isochrone, the region reachable within the threshold, and we write C for one such catchment polygon. Census tracts indexed by i partition the state. For a chosen ACS variable, each tract carries a published estimate X_i and a 90 percent margin of error M_i (U.S. Census Bureau, 2020).

Isochrone boundaries do not align with tract boundaries, so tracts at the edge of a catchment are only partly inside it. `catchmentACS` measures the overlap with coverage weights

$$w_i = \frac{a(C \cap g_i)}{a(g_i)}, \quad (1)$$

where g_i is the geometry of tract i and $a(\cdot)$ denotes area computed in an equal-area projection (EPSG:5070, a conterminous United States Albers projection). Weights lie in $(0, 1]$; intersections with coverage below 10^{-6} are treated as slivers and dropped. Geometries are validated and repaired before intersection, and an audit trail records the intersection area and tract area behind every weight. Figure 2 in Section 4 shows what these weights look like along the boundary of a real catchment.

For count variables such as total population or the number of persons below the poverty line, labeled the `spatial_total` family in the output, the catchment estimate is the coverage-weighted sum

$$\hat{X}_C = \sum_i w_i X_i. \quad (2)$$

Margins of error follow the approximation formulas that the Census Bureau recommends for user-derived estimates (U.S. Census Bureau, 2020), extended to fractional coverage weights. Writing $z = 1.645$ for the 90 percent confidence factor, the standard error of a published estimate is M_i/z , and the weighted sum in Equation (2) has margin of error

$$M_C = \sqrt{\sum_i w_i^2 M_i^2}. \quad (3)$$

For intensive variables such as per capita income, averaging is weighted by each tract's share of the intersected area rather than by the coverage fractions of Equation (1): with $v_i = a(C \cap g_i) / \sum_j a(C \cap g_j)$, the catchment mean and its margin of error are

$$\bar{X}_C = \sum_i v_i X_i, \quad M_{\bar{X}} = \sqrt{\sum_i v_i^2 M_i^2}. \quad (4)$$

Medians cannot be aggregated exactly from tract medians, so median household income is returned as an area-share weighted mean of tract medians and labeled `median_proxy` rather

than presented as a true catchment median. Appendix A lists the full variable catalog and the estimand family attached to each entry.

A derived proportion $\hat{p} = \hat{X}_{\text{num}}/\hat{X}_{\text{den}}$, whose numerator is a subset of its denominator, has

$$M_{\hat{p}} = \frac{\sqrt{M_{\text{num}}^2 - \hat{p}^2 M_{\text{den}}^2}}{\hat{X}_{\text{den}}}. \quad (5)$$

The quantity under the square root can be negative, a case the Census Bureau documentation anticipates by recommending the general ratio formula instead, which replaces the minus sign with a plus sign so that the quantity under the square root cannot be negative. **catchmentACS** exposes both forms through a dispatch argument and, by default, applies the conservative ratio formula to every derived rate; when the subset-proportion form is requested and its radicand is negative, the package falls back to the ratio formula and records the substitution in the affected row. Rates are computed from aggregation carriers: the numerator and denominator are aggregated to the catchment first, and the ratio and its margin of error are formed at the catchment level. Section 5 measures what the conservative default costs.

Two assumptions should be made explicit. First, coverage weighting inherits the standard areal-interpolation assumption that each variable is uniformly distributed within its tract (Comber & Zeng, 2019); where population is spatially concentrated, a population-weighted or dasymetric refinement would give different estimates. Second, the error formulas treat tract estimates as independent and the weights as fixed, so correlation among tract estimates and uncertainty in the isochrone geometry itself are not reflected in M_C . These caveats also accompany the Census Bureau formulas in ordinary aggregate use; the package makes them explicit by writing its assumptions into the provenance of every row. Section 5 and Appendix E examine what the second assumption costs, and we shall return to both points in Section 8.

3. The **catchmentACS** package

The package is installed from CRAN.

```
install.packages("catchmentACS")
```

All analyses in this paper use **catchmentACS** 0.5.0 with **sf** 1.1.1 and **tidycensus** 1.8.1. The pipeline is organized around six user-facing functions, each of which can also be called on its own (Table 1).

Figure 1 shows how the stages fit together, with the Birmingham example of Section 4 traced through them. Sites enter as an **sf** point table, **cacs_isochrone()** dispatches to a routing provider, and **cacs_acs_prefetch()** obtains tract estimates and margins of error through **tidycensus**, with geometries resolved against Census TIGER/Line files via **tigris**. The intersection stage computes the weights of Equation (1), the propagation stage applies

Table 1. The six core functions of catchmentACS.

Function	Role
<code>cacs_isochrone()</code>	Drive-time polygons through provider dispatch, with validation
<code>cacs_acs_prefetch()</code>	Cached, validated retrieval of single-state ACS tract tables
<code>cacs_intersect_weight()</code>	Equal-area intersection and coverage weights, with a per-tract audit
<code>cacs_propagate_moe()</code>	Weighted margin-of-error combination for each estimand family
<code>cacs_derive_rates()</code>	Curated rates with ratio margins of error and fallback tracking
<code>cacs_run()</code>	Orchestration of the full pipeline with condition capture and caching

Equations (3) to (5), and the rate stage derives five curated rates with ratio margins of error: poverty, Supplemental Nutrition Assistance Program (SNAP) receipt, Supplemental Security Income receipt, unemployment, and labor-force participation. The result is a long tibble with one row per site, drive time, and variable.

The default routing provider is OSRM, wrapped through the `osrm` package, with a budget guard that limits load on the public demonstration server and support for self-hosted instances; OpenRouteService is supported through the `openrouteservice` GitHub package and an API key. Mapbox and `r5r` appear in the provider interface as reserved names that fail immediately with an informative error instead of running partially. Every stage also accepts precomputed inputs: `precomputed_isochrones` substitutes for live routing and `acs` substitutes for the Census API, and this is how the analyses in this paper are run offline. The current release supports one state per call, ACS five-year endpoints from 2009 to 2024, area weighting, and the five curated rates; population weighting is a reserved option to which we return in Section 8.

Results are ordinary tibbles carrying 27 columns of estimates and provenance. Alongside `estimate` and `moe`, each row records which estimand family and margin-of-error formula were applied, whether a fallback occurred and why, and where a failure originated if the row could not be computed; Appendix B documents the full column contract. Run-level metadata, captured warnings, and cache state are attached as attributes, and `cacs_describe()` prints them as a readable provenance map. Expensive artifacts are cached on disk under SHA-256 fingerprints of their inputs, so that a repeated call reproduces results without recomputation and cached artifacts are invalidated whenever an input changes. A set of `leaflet` helpers (Cheng et al., 2025), `cacs_plot_site_isochrone()` through `cacs_plot_site_pipeline()`, renders interactive stage-by-stage walkthrough maps for a single site.

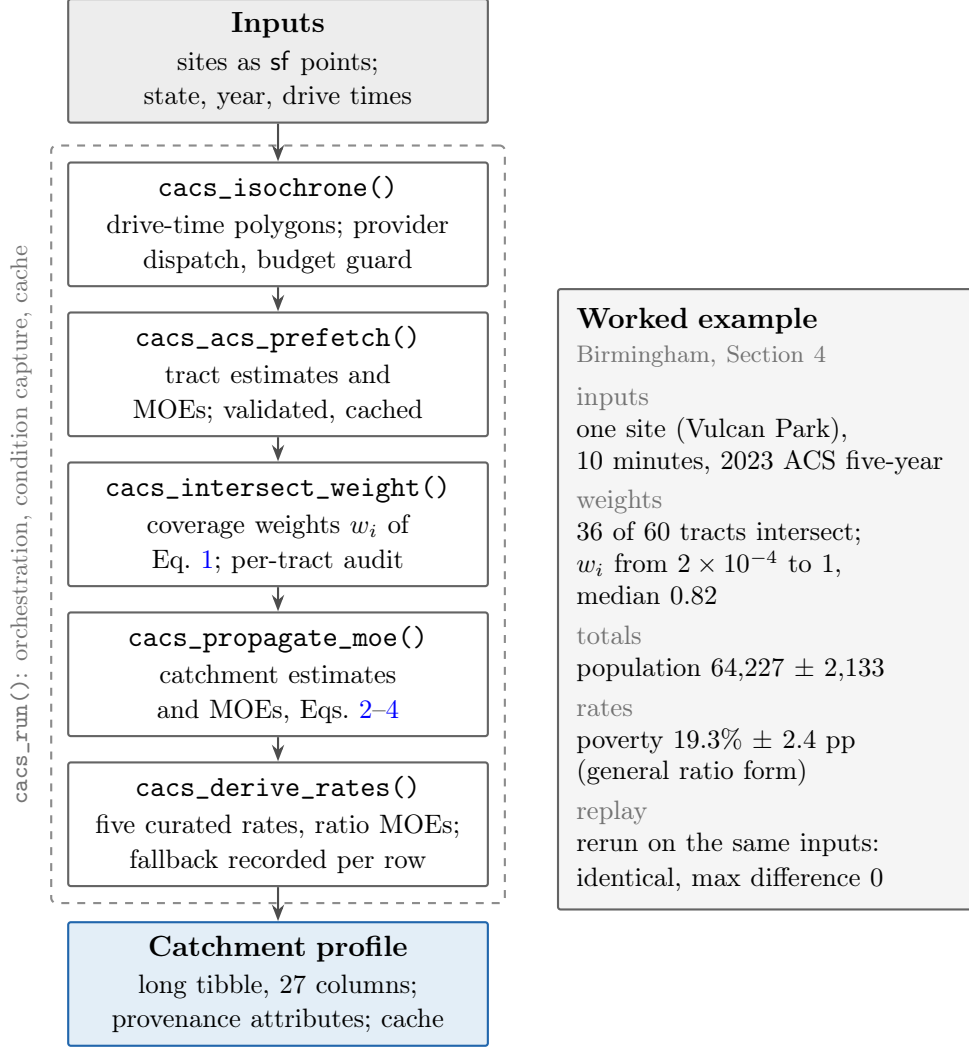


Figure 1. The catchmentACS pipeline. Grey blocks are data, white blocks are the stage functions, and the dashed envelope marks the stages orchestrated by a single call to `cacs_run()`. The right panel traces the Birmingham catchment of Section 4 through the pipeline; every number in it is computed when this document is rendered.

3.1. Design and implementation. The user we had in mind when designing the interface is an analyst in a policy office who must produce the same profile every year for a changing list of sites, and whose results are subject to audit. A single call therefore carries defaults for everything a routine run does not need to decide, while the six stage functions of Table 1 remain individually callable, so that any intermediate object, in particular the per-tract weight audit examined in Section 4, can be inspected without reassembling the pipeline by hand. Conditions raised inside a stage are captured and returned with the results instead of being thrown, and failure is isolated to the site and variable in which it arose, so that one unroutable address does not abort a batch of several hundred.

The package uses S3 classes throughout, and the object model was kept deliberately thin. `cacs_run()` returns a tibble with the additional class `cacs_run_result`, so that the ordinary `dplyr` and `ggplot2` verbs apply to it unchanged and a consumer who is unaware of the class loses nothing but the provenance. That provenance rides in attributes rather than in a bespoke container, and methods for `print()`, `summary()`, `as_tibble()`, and `dplyr::group_by()` are supplied so that the class behaves predictably under the verbs an analyst is most likely to reach for. `cacs_describe()` is a generic that renders the attributes of whichever stage output it is given; Appendix C shows its output for the batch workload.

Dependencies were kept to what the aggregation itself requires. The package imports thirteen packages, of which `sf`, `tidycensus`, and `tigris` do the spatial and retrieval work and the remainder are small infrastructure libraries for messaging, hashing, caching, and tidy data manipulation. All four routing back ends are in `Suggests` rather than `Imports`, as is `leaflet`, so that a user who supplies precomputed isochrones installs no routing stack at all. This is the reason the analyses in this paper require neither a routing account nor the packages that would talk to one.

The package is not parallelized, and we do not expect it to benefit from being so. The aggregation cost is linear in the number of sites multiplied by the number of tracts each catchment intersects, and it is small; profiling the batch of Section 6 requires about one second. In a live run the wall-clock time is dominated instead by the routing service, which returns one isochrone per request and is rate limited, and the budget guard on the public OSRM demonstration server exists to keep the package from issuing requests faster than that server is meant to receive them. Parallelizing the R side would therefore move no bottleneck and would risk violating a provider’s terms. Memory scales with the tract panel for the state being profiled rather than with the number of sites, since the panel is retrieved once and reused across the batch.

The analyses in this paper can be reproduced without a Census API key or a routing service account, since every computation runs from data bundled with the package. Static figures are drawn with `ggplot2` (Wickham, 2016) and `patchwork` (Pedersen, 2025).

4. A single catchment in Birmingham

We begin with a single catchment based on real data. The package ships a fixture, built around a site at Vulcan Park in Birmingham, Alabama, that contains a genuine 10-minute OSRM drive-time isochrone, 60 surrounding Jefferson County census tracts, and published 2023 ACS five-year estimates with margins of error for the 14 variables in the default catalog.

```
library(catchmentACS)
library(sf)
library(dplyr)
```



```

vw <- readRDS(system.file("extdata", "visual_walkthrough_fixture.rds",
                           package = "catchmentACS"))
bhm <- cacs_run(sites = vw$sites_df, state = "AL", year = 2023,
               drive_times = 10,
               precomputed_isochrones = vw$iso_sf, acs = vw$acs_sf)
as_tibble(bhm) |> print(n = 6)

#> # A tibble: 19 x 27
#>   site_id drive_time_min ring_topology variable estimate moe weight_sum
#>   <chr>      <int> <chr>      <chr>      <dbl> <dbl> <dbl>
#> 1 AL_BHM_01      10 cumulative labor_fo~ 6.49e-1 3.43e-2 23.3
#> 2 AL_BHM_01      10 cumulative poverty_~ 1.93e-1 2.43e-2 23.3
#> 3 AL_BHM_01      10 cumulative snap_rate 1.12e-1 1.32e-2 23.3
#> 4 AL_BHM_01      10 cumulative ssi_rate 5.04e-2 8.43e-3 23.3
#> 5 AL_BHM_01      10 cumulative unemp_ra~ 4.24e-2 9.24e-3 23.3
#> 6 AL_BHM_01      10 cumulative B01003_0~ 6.42e+4 2.13e+3 23.3
#> # i 13 more rows
#> # i 20 more variables: n_tracts <int>, n_tracts_num <int>,
#> #   n_tracts_den <int>, provider <chr>, profile <chr>,
#> #   osm_snapshot_date <chr>, acs_year <int>, weight_method <chr>,
#> #   estimand_family <chr>, weight_basis <chr>, moe_formula_requested <chr>,
#> #   moe_formula_effective <chr>, moe_fallback <lgl>,
#> #   moe_fallback_reason <chr>, failure_origin <chr>, ...

```

Of the 60 tracts in the fixture, 36 intersect the isochrone. The per-tract audit behind the coverage weights is available directly from the intersection stage.

```

iw <- cacs_intersect_weight(vw$iso_sf, vw$acs_sf, keep_tract_audit = TRUE)
audit <- attr(iw, "cacs_tract_audit")
summary(audit$area_wt)

```

```

#>      Min.    1st Qu.    Median      Mean   3rd Qu.      Max.
#> 0.0002148 0.2577366 0.8192452 0.6485008 1.0000000 1.0000000

```

The weights range from 2×10^{-4} , for a tract that barely overlaps the catchment, to 1 for tracts that are fully contained, with a median of 0.82. Figure 2 shows the catchment, the weights, and, in the enlarged panel, the values that Equation (1) assigns along the boundary, where the isochrone cuts tracts into fractions.

The resulting profile estimates 64,227 residents in the catchment, with a margin of error of 2,133. The poverty universe contains 58,947 persons, of whom $11,403 \pm 1,373$ are below the poverty line, and the derived poverty rate is 19.3 percent with a margin of error of 2.4 percentage points (Table 2). Median household income is reported as $68,803 \pm 5,658$

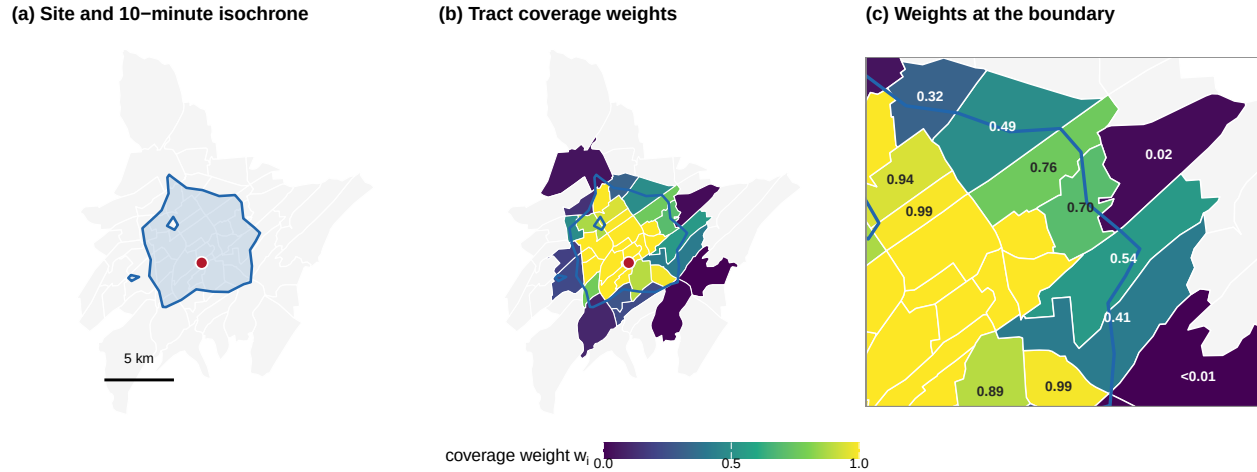


Figure 2. The Birmingham example. Panel (a) shows the site and its 10-minute drive-time isochrone over the 60 Jefferson County tracts in the fixture. Panel (b) shades the 36 intersecting tracts by their coverage weight from Equation (1); tracts outside the catchment are unshaded. Panel (c) enlarges the north-eastern boundary and prints the computed weight on each partly covered tract.

dollars and is labeled `median_proxy` in the output, which keeps the approximation visible to downstream users. The last column of Table 2 records the conservative default dispatch discussed in Section 2.

Table 2. Derived rates for the Birmingham catchment, with 90 percent margins of error.

variable	estimate	moe	moe_formula_effective
labor_force_participation	0.649	0.034	general_ratio_conservative
poverty_rate	0.193	0.024	general_ratio_conservative
snap_rate	0.112	0.013	general_ratio_conservative
ssi_rate	0.050	0.008	general_ratio_conservative
unemp_rate	0.042	0.009	general_ratio_conservative

Rerunning the pipeline on the same inputs reproduces its results exactly: the fixture stores the profile computed when it was built, and a fresh call to `cacs_run()` returns the same values.

```
replay <- inner_join(
  as_tibble(bhm) |> select(variable, estimate, moe),
  as_tibble(vw$run_result) |>
    select(variable, ref_estimate = estimate, ref_moe = moe),
  by = "variable")
max(abs(replay$estimate - replay$ref_estimate),
  abs(replay$moe - replay$ref_moe))
```

```
#> [1] 0
```

In a session with credentials, the same call runs end to end from live services, with routing by OSRM and retrieval through the Census API.

```
tidycensus::census_api_key("YOUR_KEY")
profiles <- cacs_run(sites = my_sites, state = "AL", year = 2023,
  drive_times = c(5, 10, 15), provider = "osrm")
```

5. Checking the propagated margins of error

The output of the pipeline consists of point estimates and margins of error, and the two halves call for different checks. For the point estimates, the aggregation in Equation (2) is standard areal-weighted interpolation, so `catchmentACS` can be compared directly with the general-purpose implementation in `sf`. Interpolating the three extensive inputs behind the poverty profile of the Birmingham catchment with `st_interpolate_aw()` reproduces the package’s values to machine precision, with a maximum relative difference of about 10^{-14} ; the code for this comparison is given in Appendix C, and Table 3 shows the result. Both implementations compute the same coverage-weighted sums under the same uniform-distribution assumption, so exact agreement is expected. The interpolated layer, however, carries no uncertainty columns, a point to which we return in Section 7.

Table 3. Agreement between `catchmentACS` and `sf::st_interpolate_aw()` on the extensive inputs of the Birmingham poverty profile. The point estimates agree to machine precision; only `catchmentACS` carries margins of error.

Quantity	<code>st_interpolate_aw</code>	<code>catchmentACS</code>	<code>catchmentACS</code> MOE
Total population	64,227	64,227	2,133
Poverty universe	58,947	58,947	2,112
Below poverty	11,403	11,403	1,373

The margins of error cannot be checked the same way, because no general-purpose implementation produces them. Equation (3) is exact for a weighted sum of independent estimates, but Equation (5) and the general ratio form are approximations, and the package applies the more conservative of the two by default. How much is given up by that choice cannot be read off the formulas, so we examine it by simulation.

Under the assumptions stated in Section 2 the sampling distribution of a catchment rate can be simulated directly. Write A_i for the persons below the poverty line in tract i and B_i for the poverty universe, with published margins M_i^A and M_i^B , and decompose $B_i = A_i + C_i$, where C_i counts the persons at or above the poverty line. Equation (5) is the delta-method approximation that follows when A_i and C_i are treated as independent, so we simulate on that decomposition, drawing $A_i \sim N(A_i, (M_i^A/z)^2)$ and $C_i \sim N(B_i - A_i, (M_i^B/z)^2 - (M_i^A/z)^2)$

independently across tracts and recomputing the weighted aggregation for each of 20,000 replicates. Note that this exercise checks the propagation machinery against the assumptions from which it is derived; it does not validate those assumptions, which are taken up at the end of this section. The full simulation code, which runs when this document is rendered, is listed in Appendix D.

Table 4. Margins of error for the Birmingham catchment poverty rate, with the coverage each nominally 90 percent interval attains against the simulated sampling distribution.

Quantity	MOE (pp)	Coverage
Subset-proportion form, Equation (5)	2.22	0.939
General ratio form (package default)	2.43	0.961
Simulation, 20,000 replicates	1.96	0.900

The margin of error that the package reports for this catchment, 2.43 percentage points, is recovered exactly by recomputing the general ratio form from the published tract values, which confirms that the implementation applies the formula it documents. Table 4 sets the two published forms beside the simulated sampling variability, and Figure 3 draws the comparison. Both published forms are conservative here, and the general ratio form applied by default is the more conservative of the two: its interval is about 24 percent wider than the simulated variability and covers 96.1 percent in place of the nominal 90. The widening is the cost of the guarantee that the radicand cannot become negative, and it applies to every derived rate. An analyst who prefers the narrower interval can request the subset-proportion form through the dispatch argument, accepting that the package will fall back to the ratio form, and record the substitution, on any row where the radicand turns negative.

The same exercise was repeated for all five curated rates, and the results, tabulated in Appendix D, show that the conservatism of the default grows with the level of the rate. For the three smallest rates the two forms nearly coincide and the default covers between 91 and 94 percent, whereas for labor-force participation, at 65 percent the largest of the five, the default interval is 2.4 times the simulated width and covers essentially always. The reason is visible in the formulas: the two forms differ by $2\hat{p}^2 M_{\text{den}}^2$ under the square root, so the gap grows with the square of the rate.

The independence assumption behind these calculations remains to be examined, and its consequences differ by estimand. For a weighted total, positive correlation among tract estimates unambiguously widens the true margin of error. If the tract estimates carry a common correlation ρ , the variance of the weighted sum in Equation (2) becomes

$$\text{Var}(\hat{X}_C) = \sum_i w_i^2 s_i^2 + \rho \left[\left(\sum_i w_i s_i \right)^2 - \sum_i w_i^2 s_i^2 \right], \quad (6)$$

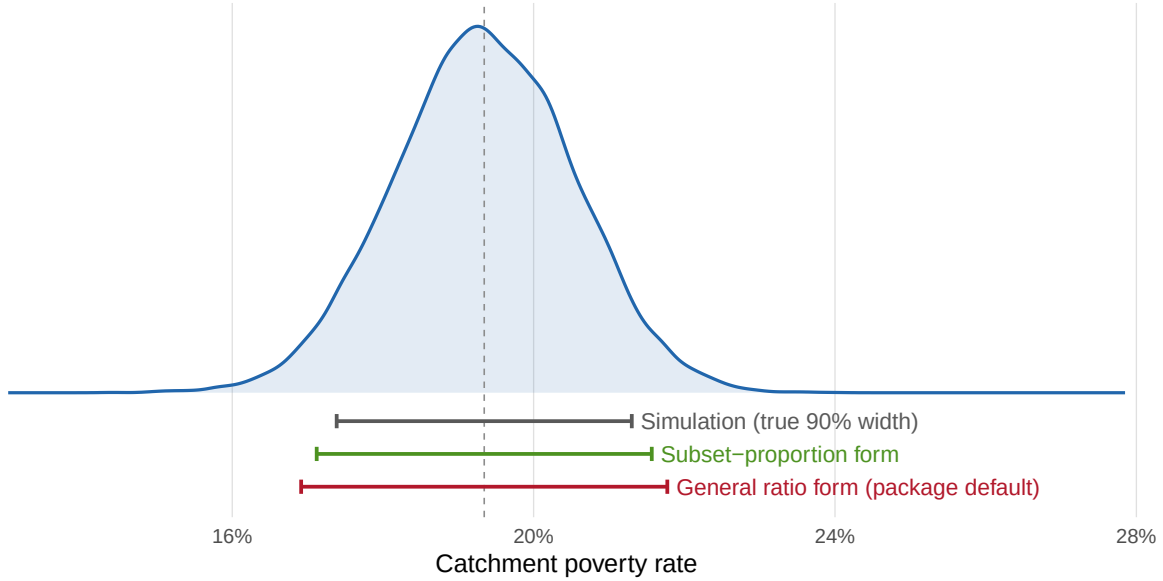


Figure 3. Simulated sampling distribution of the Birmingham catchment poverty rate over 20,000 replicates, drawn under the independence assumption of Section 2, with the half-widths of the three nominally 90 percent intervals of Table 3 beneath it. Both published forms are wider than the simulated variability, and the general ratio form used by default is the wider of the two.

where $s_i = M_i/z$, and since a catchment pools several dozen tracts the cross-product term dominates. For the total population of the Birmingham catchment, a correlation of $\rho = 0.05$ would inflate the reported margin of error by a factor of 1.47, and $\rho = 0.10$ by a factor of 1.82 (Table 5). ACS tract estimates within a county are built from a common sample, so ρ is positive rather than zero, and the margins of error that the package reports for totals should be read as understatements of unknown size. The published margins themselves do not identify ρ , and the approximation formulas that the Census Bureau recommends for user-derived estimates make the same independence assumption in ordinary aggregate use. Folch et al. (2023) showed that the covariance between ACS estimates can be recovered empirically from the Bureau’s variance replicate tables, which republish each estimate under 80 replicate weightings, and that the replicate route outperforms both the analytic formulas and independent simulation; the tables cover a selected set of detailed tables, and consuming them is an extension to which we return in Section 8.

For a derived rate the picture is different, because correlated errors that inflate the numerator and the denominator together largely cancel in their ratio. In Appendix E we simulate the twenty-site comparison of Section 6 under two bracketing correlation structures, one in which a catchment-wide factor moves both components of the rate and one in which the components move independently, with ρ up to 0.3. Under both structures the number of site pairs that cannot be distinguished stays within a few pairs of the count reported in Section 6, partly because the cancellation just described limits the damage and partly because the

Table 5. Margin of error for the total population of the Birmingham catchment under a common correlation among the tract estimates. The first row is the value the package reports.

Correlation	Total-population MOE	Inflation
0.00	2,133	1.00x
0.05	3,132	1.47x
0.10	3,881	1.82x
0.20	5,058	2.37x

conservative ratio margins are wide enough to absorb what remains. The comparisons of Section 6 are therefore not fragile to the independence assumption, although the individual margins for totals are.

6. Profiles for twenty sites

In applications, profiles are usually required for many sites at once. The second bundled fixture therefore supports a batch workflow: 20 synthetic Alabama site locations, 60 precomputed isochrones forming cumulative 5-, 10-, and 15-minute rings, and a synthetic 911-tract ACS panel for the full default catalog. The panel is synthetic but constructed to mimic production ACS data, including the missing-data sentinels that the Census API returns for suppressed tract estimates, and it also serves as the package’s frozen regression baseline: continuous integration reruns this workload and checks the results against a stored reference output within small numerical tolerances. Appendix C maps the workload and prints its provenance. We profile all 20 sites at the three thresholds in one call.

```
sites <- readRDS(system.file("extdata", "legacy_2025_sites.rds",
                             package = "catchmentACS"))
iso <- readRDS(system.file("extdata", "legacy_2025_isochrones.rds",
                             package = "catchmentACS"))
acs <- readRDS(system.file("extdata", "sample_alabama_subset.rds",
                             package = "catchmentACS"))
timing <- system.time(
  profiles <- cacs_run(sites = sites, state = "AL", year = 2023,
                       drive_times = c(5, 10, 15),
                       precomputed_isochrones = iso, acs = acs)
)
nrow(profiles)
```

```
#> [1] 1140
```

The full batch of 1,140 rows (20 sites, 3 drive times, 14 variables, and 5 rates) completes in 0.9 seconds on an ordinary laptop, and repeated calls return from the cache. About 5

percent of the tract estimates in the synthetic panel are missing, mirroring suppressed small-area values in production ACS data. Missing tract inputs propagate to missing catchment aggregates instead of being dropped from the weighted sums, and a rate whose aggregated numerator or denominator is unavailable is returned as NA with its origin labeled, so that the composition of an aggregate is never altered without indication.

```
as_tibble(profiles) |>
  filter(failure_origin != "none") |>
  count(variable, failure_origin)
```

```
#> # A tibble: 5 x 3
#>   variable      failure_origin     n
#>   <chr>         <chr>         <int>
#> 1 labor_force_participation carrier         6
#> 2 poverty_rate      carrier         5
#> 3 snap_rate         carrier        12
#> 4 ssi_rate          carrier        12
#> 5 unemp_rate        carrier        17
```

Table 6 lists the eight highest-poverty catchments at the 10-minute threshold, and Figure 4 shows all 18 sites with computable 10-minute poverty rates together with their margins of error. The estimates span 5.2 to 60.2 percent, and the intervals overlap heavily through the middle of the ranking.

Table 6. The eight highest 10-minute catchment poverty rates among the twenty synthetic sites, with 90 percent margins of error in percentage points.

Rank	Site	Poverty rate	MOE (pp)
1	AL_SITE_07	60.2%	9.8
2	AL_SITE_08	42.8%	12.5
3	AL_SITE_01	37.7%	5.5
4	AL_SITE_13	37.2%	7.6
5	AL_SITE_17	33.5%	5.9
6	AL_SITE_18	27.7%	4.0
7	AL_SITE_03	26.1%	5.6
8	AL_SITE_15	24.1%	5.1

The margins of error have direct consequences for comparisons between sites. Under the Census Bureau’s rule for comparing two estimates, sites j and k differ at the 90 percent level when $|\hat{p}_j - \hat{p}_k|$ exceeds $1.645\sqrt{SE_j^2 + SE_k^2}$ (U.S. Census Bureau, 2020). The rule treats the two estimates as independent, which for catchment profiles requires that the catchments do not share tracts, and in this batch they do not: no two of the twenty isochrones intersect at any threshold, as the following check confirms. Where catchments do overlap, the shared

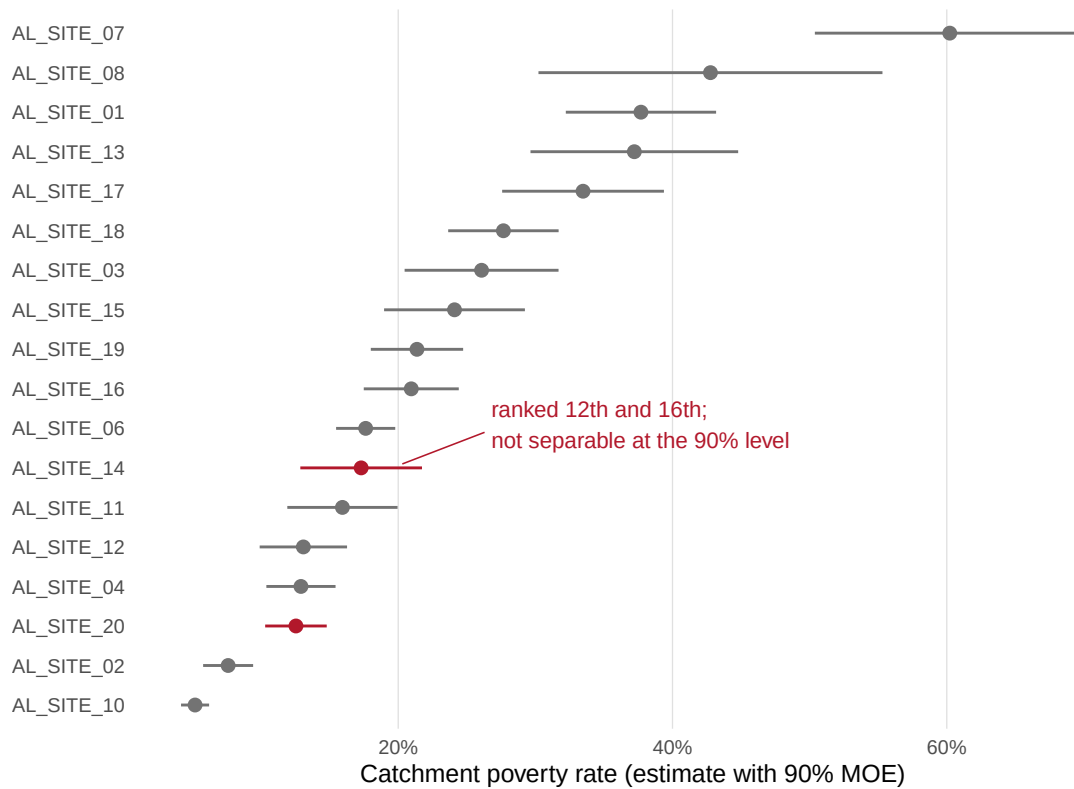


Figure 4. Ten-minute catchment poverty rates for the twenty-site batch, ordered by point estimate, with 90 percent margin-of-error intervals. Two sites with missing inputs are excluded. The highlighted pair, examined in the text, is four ranks apart yet cannot be separated at the 90 percent level.

tracts induce positive correlation between the two estimates, the standard error of their difference is smaller than the expression above, and the rule becomes conservative.

```
i10 <- iso |> filter(drive_time_min == 10) |> st_transform(5070) |>
  st_make_valid()
touching <- st_intersects(i10, i10, sparse = FALSE)
diag(touching) <- FALSE
sum(touching) / 2

#> [1] 0
```

Applying the rule to every pair of sites indicates how much of the apparent ranking is statistically supported.

```
pov10 <- as_tibble(profiles) |>
  filter(variable == "poverty_rate", drive_time_min == 10,
    !is.na(estimate)) |>
  arrange(desc(estimate)) |>
  mutate(se = moe / 1.645)
```

```
pr <- t(combn(nrow(pov10), 2))
zst <- abs(pov10$estimate[pr[, 1]] - pov10$estimate[pr[, 2]]) /
  sqrt(pov10$se[pr[, 1]]^2 + pov10$se[pr[, 2]]^2)
tibble(pairs = nrow(pr), indistinct = sum(zst < 1.645),
  share = mean(zst < 1.645))

#> # A tibble: 1 x 3
#>   pairs indistinct share
#>   <int>      <int> <dbl>
#> 1    153         33 0.216
```

About one in five pairwise comparisons cannot be distinguished at this level, and Figure 5 shows where in the ranking the indistinct pairs sit. Most lie in a band along the diagonal, between sites of neighboring rank, but the band is not uniform: it is wide where estimates are close and margins are large, and it pinches shut where a gap in the estimates happens to coincide with small margins. The indistinct pairs are also not confined to immediate neighbors. For instance, AL_SITE_14, ranked 12th at 17.3 percent, and AL_SITE_20, ranked 16th at 12.5 percent, cannot be separated at the 90 percent level, so a funding formula that treated the point ranking as exact would separate these two communities on evidence that does not support the distinction. This information is unavailable when only interpolated point estimates are reported.

Two qualifications should be attached to the count. The comparisons are not independent of one another and no adjustment for their multiplicity has been made, so the share describes how much of this ranking is separable pair by pair and is not a test of the ranking as a whole. The count is, however, robust to the independence assumption examined in Section 5: under both correlation structures considered in Appendix E, with ρ as high as 0.3, the number of indistinct pairs moves between 27 and 31 against the 33 reported here.

Since the bundled isochrones are cumulative rings, the same output also shows how profiles respond to the catchment definition itself (Figure 6). Point estimates shift as rings widen and annex different tracts, while the median margin of error falls from 4.8 percentage points at 5 minutes to 3.3 at 15 minutes as catchments pool more tracts. Sensitivity of this kind suggests that profiles should be reported for several thresholds.

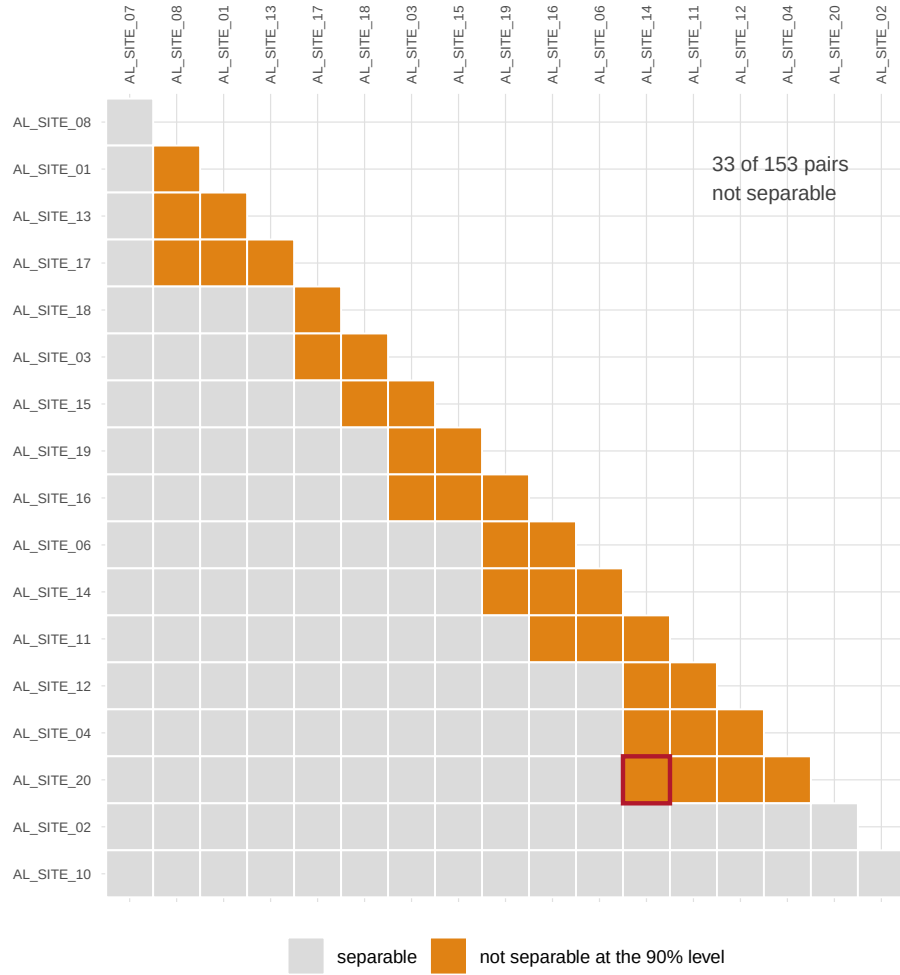


Figure 5. Pairwise comparisons of the ten-minute catchment poverty rates, with sites ordered by point estimate from the highest (top left) to the lowest. Shaded cells mark the 33 of 153 pairs that cannot be separated at the 90 percent level; the outlined cell is the pair highlighted in Figure 4. Indistinct pairs concentrate along the diagonal but are not confined to it.

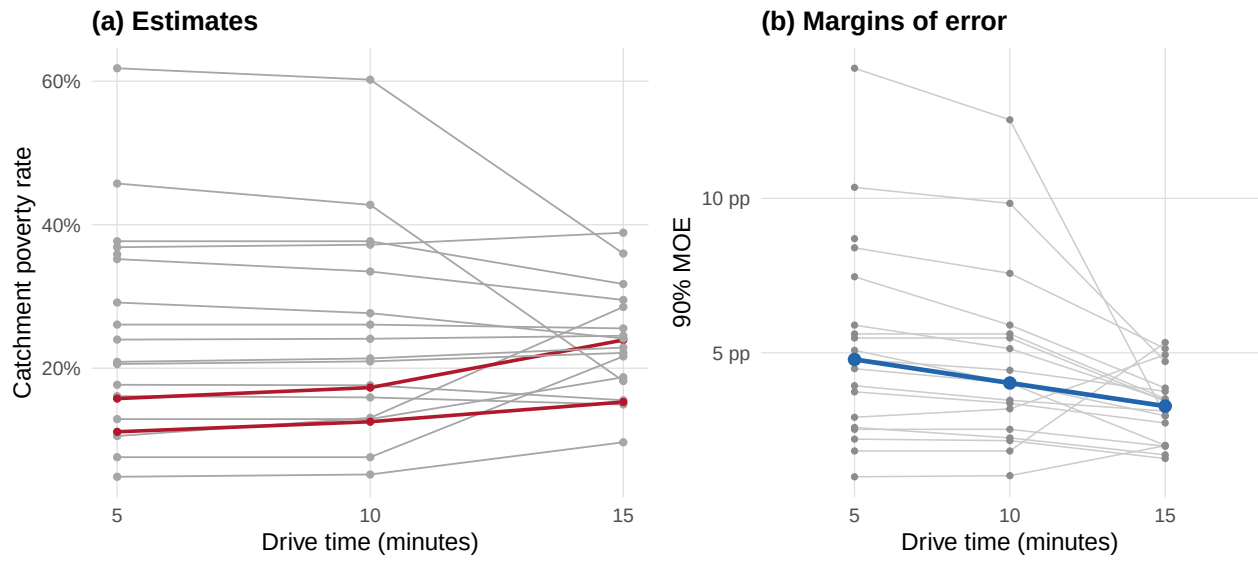


Figure 6. Catchment poverty rates and their margins of error across the 5-, 10-, and 15-minute cumulative rings. In panel (a), lines connect the same site, with the pair from Figure 4 highlighted; estimates move as widening rings change the tract composition. In panel (b), the same sites' margins of error narrow as catchments pool more tracts; the heavy line is the median.

7. Comparison with general-purpose areal interpolation

Section 5 established that `catchmentACS` and `st_interpolate_aw()` in `sf` agree to machine precision on point estimates, so the tools do not differ in what they compute but in what they attach to it. The interpolated layer returned by `st_interpolate_aw()` carries the estimate columns and nothing else, whereas every `catchmentACS` row arrives with a margin of error, the formula that produced it, and its fallback status. The same contrast holds for the other general-purpose tools. The `interpolate_pw()` function in `tidycensus` refines the weights with a population surface, which relaxes the uniformity assumption, but as of `tidycensus` 1.8.1 it returns interpolated estimates without margins of error. `tidycensus` does provide the Census Bureau error calculators (`moe_sum()`, `moe_prop()`, and `relatives`), but these implement the unweighted formulas for whole-unit combinations and do not accommodate the fractional coverage weights of Equation (1), so they cannot be applied to a catchment aggregation without rederiving the weighted forms. `areal` likewise focuses on interpolated estimates. Table 7 summarizes these differences.

Table 7. Division of labor between general-purpose areal interpolation tools and `catchmentACS`. Catalog means that the package aggregates a fixed catalog of ACS variables rather than arbitrary user-supplied columns. Reserved means that the option is present in the interface but fails with an informative error pending a future release.

Capability	<code>st_interpolate_aw</code>	<code>areal</code>	<code>interpolate_pw</code>	<code>catchmentACS</code>
Areal interpolation to overlapping polygons	yes	yes	yes	yes
Arbitrary user-supplied input variables	yes	yes	yes	catalog
Population-weighted interpolation	no	no	yes	reserved
Margin-of-error propagation	no	no	no	yes
Derived rates with ratio MOEs	no	no	no	yes
Multi-site, multi-threshold orchestration	no	no	no	yes
Per-row provenance and caching	no	no	no	yes

The second row of Table 7 records the direction in which the general-purpose tools are the more capable. They interpolate whatever numeric columns they are given, from any source, whereas `catchmentACS` aggregates the 14 variables of its default ACS catalog (Appendix A), since the margin-of-error formulas it applies depend on knowing which estimand family each variable belongs to and which carriers form each rate. An analyst whose variables lie outside the catalog, or whose data are not ACS estimates at all, is better served by the general-purpose functions.

`catchmentACS` is designed to complement these tools: it wraps `tidycensus` for retrieval and `sf` for geometry, and an analyst who requires a single interpolated count is well served by the general-purpose functions. The package is intended for settings in which the same pipeline must be run for many sites and several thresholds, reproduced exactly under audit, and

accompanied by uncertainty measures that indicate whether observed differences between communities are statistically supported.

8. Discussion

We have described an R package that assembles drive-time catchment profiles of ACS characteristics with margins of error propagated from published tract values to catchment estimates and derived rates, together with per-row provenance recording how every number was formed. The point-estimate half of the calculation was verified against `st_interpolate_aw()` to machine precision, and the margin-of-error half was checked by simulation against the assumptions from which it is derived. This revealed that the conservative ratio formula applied by default attains about 96 percent coverage in place of the nominal 90 for the poverty rate, that its conservatism grows with the square of the rate, and that the pairwise site comparisons it supports are robust to the independence assumption under both of the correlation structures we examined. In the twenty-site case study, about a fifth of the pairwise site comparisons had no statistical support at the 90 percent level, a fact that could not have been established from interpolated point estimates alone.

Several scope decisions in the current release should be borne in mind. Calls cover one state at a time, so multi-state studies loop over states. Weighting is by area only. Unfortunately, the population-weighted option is not yet implemented; it is present in the interface but reserved, and until it is released, `interpolate_pw()` in [tidycensus](#) remains the appropriate tool when a population surface should refine point estimates, at the cost of forgoing propagated margins of error. The rate catalog is fixed at the five curated rates, the aggregation catalog at the 14 default variables, medians are aggregated as labeled proxies, and two of the four provider names, Mapbox and [r5r](#), are reserved names that fail with an informative error.

The margins of error inherit the Census Bureau approximation framework, and Section 5 measured the cost of its two assumptions. The two act in opposite directions, and their net effect depends on the estimand. For totals, the independence assumption is the serious one: because a catchment pools several dozen tracts drawn from a common sample, mild positive correlation among the tract estimates would raise the reported margins appreciably, and the package, following the Bureau's own recommended formulas, does not attempt to reflect it. For derived rates, the conservative ratio default widens the intervals while correlated errors partially cancel in the ratio, and in the twenty-site experiment of Appendix E the reported comparisons were stable under every correlation structure we tried. Margins for totals should therefore be read as understatement of unknown size, while the rate comparisons that motivated the package rest on firmer ground. The uniform-distribution assumption of areal

interpolation applies as well, and its magnitude has not been quantified here; a comparison against a population-weighted surface is the natural next check, and it awaits the reserved weighting option. Isochrones themselves depend on the routing engine and the underlying OpenStreetMap snapshot, which is why the provenance columns record the provider, profile, and snapshot date of every polygon. Finally, the twenty-site case study rests on a synthetic panel built to mimic production ACS data, so its numbers illustrate what the propagated margins of error do to a ranking rather than describe any real set of communities.

Several extensions are planned. Population and dasymetric weighting for the intersection stage would relax the uniformity assumption, multi-state calls and activation of the reserved providers would widen the range of applications, and user-defined rate catalogs would open the carrier and fallback machinery to arbitrary ratios. The margin-of-error framework extends unchanged to each of these settings, since each is a change in how weights or carriers are constructed rather than in how uncertainty is combined. The independence assumption is the one place where a further design-based improvement is available: the Census Bureau’s variance replicate tables republish selected tables under 80 replicate weightings, and Folch et al. (2023) showed that aggregating the replicates recovers the covariance that the published margins omit. Aggregating replicate estimates over the coverage weights of Equation (1) would yield catchment margins of error that reflect the correlation among tracts, at the cost of restricting the catalog to the tables for which replicates are published, and it is the extension we consider most valuable. A model-based alternative in the spirit of Spielman and Folch (2015) would also address the assumption, at the cost of departing from the published margins of error, which the auditable pipelines that motivated this package are required to report unaltered.

All analyses in this paper run from data bundled with the package and complete in minutes; neither a Census API key nor a routing account is required. The package is available on CRAN, with source code at <https://github.com/joonho112/catchmentACS> and documentation, including eleven vignettes covering theory, practice, and migration between releases, at <https://joonho112.github.io/catchmentACS/>.

Software and reproducibility. `catchmentACS` is open-source software released under the MIT license (project home page <https://github.com/joonho112/catchmentACS>; documentation at <https://joonho112.github.io/catchmentACS/>), requiring R ($\geq 4.1.0$) and containing no compiled code. This paper is an R Markdown document with live code chunks: every number, table, and figure it reports, including the numbers printed inside Figure 1 and the simulations of Section 5 and Appendices D and E, is computed when the document is rendered, from data bundled with the package, so no result is transcribed by hand. The source,

together with a standalone script that reproduces every reported number outside the document, is archived with the preprint. All analyses were run with R version 4.6.0 (2026-04-24) and `catchmentACS` 0.5.0, `sf` 1.1.1, and `tidycensus` 1.8.1.

Data availability. Both fixtures used in this paper are distributed with the package. The Birmingham fixture contains published 2023 ACS five-year estimates for census tracts in Jefferson County, Alabama, which are public data, together with an OSRM drive-time isochrone for a single point at a public landmark. The twenty-site fixture is synthetic, as described in Section 6 and Appendix C: its site coordinates lie on a generated grid and correspond to no real facility, and its tract panel carries placeholder identifiers rather than Census geographies. No restricted data, and no data describing individuals, were used at any stage.

Competing interests. The author declares that he has no competing interests.

Funding. This work received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Use of large language models. The author used Claude, a large language model developed by Anthropic, as a coding assistant and for minor sentence-level editing. Everything else, including the design of the package and of the study, the implementation of the estimators, the analyses, the interpretation of the results, and the substance of the manuscript, is the author’s own work, and the author takes full responsibility for the content of this paper.

References

- Cheng, J., Schloerke, B., Karambelkar, B., Xie, Y., & Aden-Buie, G. (2025). *leaflet: Create interactive web maps with the JavaScript 'Leaflet' library* [R package version 2.2.3]. <https://doi.org/10.32614/CRAN.package.leaflet>
- Comber, A., & Zeng, W. (2019). Spatial interpolation using areal features: A review of methods and opportunities using new forms of data with coded illustrations. *Geography Compass*, 13(10), e12465. <https://doi.org/10.1111/gec3.12465>
- Folch, D. C., Arribas-Bel, D., Koschinsky, J., & Spielman, S. E. (2016). Spatial variation in the quality of American Community Survey estimates. *Demography*, 53(5), 1535–1554. <https://doi.org/10.1007/s13524-016-0499-1>
- Folch, D. C., Spielman, S., & Graber, M. (2023). The impact of covariance on American Community Survey margins of error: Computational alternatives. *Population Research and Policy Review*, 42(4), 55. <https://doi.org/10.1007/s11113-023-09786-y>
- Giraud, T. (2022). osrm: Interface between R and the OpenStreetMap-based routing service OSRM. *Journal of Open Source Software*, 7(78), 4574. <https://doi.org/10.21105/joss.04574>

- Glenn, E. H. (2019). *acs: Download, manipulate, and present American Community Survey and decennial data from the US Census* [R package version 2.1.4; archived on CRAN]. <https://CRAN.R-project.org/package=acs>
- Jurjevich, J. R., Griffin, A. L., Spielman, S. E., Folch, D. C., Merrick, M., & Nagle, N. N. (2018). Navigating statistical uncertainty: How urban and regional planners understand and work with American Community Survey (ACS) data for guiding policy. *Journal of the American Planning Association*, 84(2), 112–126. <https://doi.org/10.1080/01944363.2018.1440182>
- Lee, J. (2026). *catchmentACS: Isochrone-based area-weighted ACS aggregation* [R package version 0.5.0]. <https://doi.org/10.32614/CRAN.package.catchmentACS>
- Luo, W., & Wang, F. (2003). Measures of spatial accessibility to health care in a GIS environment: Synthesis and a case study in the Chicago region. *Environment and Planning B: Planning and Design*, 30(6), 865–884. <https://doi.org/10.1068/b29120>
- Pebesma, E. (2018). Simple features for R: Standardized support for spatial vector data. *The R Journal*, 10(1), 439–446. <https://doi.org/10.32614/RJ-2018-009>
- Pedersen, T. L. (2025). *patchwork: The composer of plots* [R package version 1.3.2]. <https://doi.org/10.32614/CRAN.package.patchwork>
- Pereira, R. H. M., & Herszenhut, D. (2026). *accessibility: Transport accessibility measures* [R package version 1.5.0]. <https://doi.org/10.32614/CRAN.package.accessibility>
- Pereira, R. H. M., Saraiva, M., Herszenhut, D., Braga, C. K. V., & Conway, M. W. (2021). r5r: Rapid realistic routing on multimodal transport networks with R5 in R. *Findings*. <https://doi.org/10.32866/001c.21262>
- Prener, C., & Revord, C. (2019). areal: An R package for areal weighted interpolation. *Journal of Open Source Software*, 4(37), 1221. <https://doi.org/10.21105/joss.01221>
- Spielman, S. E., Folch, D., & Nagle, N. (2014). Patterns and causes of uncertainty in the American Community Survey. *Applied Geography*, 46, 147–157. <https://doi.org/10.1016/j.apgeog.2013.11.002>
- Spielman, S. E., & Folch, D. C. (2015). Reducing uncertainty in the American Community Survey through data-driven regionalization. *PLOS ONE*, 10(2), e0115626. <https://doi.org/10.1371/journal.pone.0115626>
- U.S. Census Bureau. (2020). Understanding and using American Community Survey data: What all data users need to know. <https://www.census.gov/programs-surveys/acs/library/handbooks/general.html>
- Walker, K. (2023). *Analyzing US census data: Methods, maps, and models in R*. Chapman; Hall/CRC. <https://walker-data.com/census-r/>
- Walker, K. (2025). *tigris: Load census TIGER/Line shapefiles* [R package version 2.2.1]. <https://doi.org/10.32614/CRAN.package.tigris>

- Walker, K. (2026). *mapboxapi: R interface to 'Mapbox' web services* [R package version 0.6.3].
<https://doi.org/10.32614/CRAN.package.mapboxapi>
- Walker, K., & Herman, M. (2026). *tidycensus: Load US census boundary and attribute data as 'tidyverse' and 'sf'-ready data frames* [R package version 1.8.1]. <https://doi.org/10.32614/CRAN.package.tidycensus>
- Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer-Verlag New York.
<https://ggplot2.tidyverse.org>

Online Supplemental Material

catchmentACS: Isochrone-Based Catchment-Area Profiles from American Community Survey Data with Margin-of-Error Propagation

Appendix A lists the default variable catalog and the estimand family and margin-of-error treatment attached to each entry. Appendix B documents the 27-column output contract. Appendix C describes the twenty-site batch workload of Section 6 and prints its provenance. Appendix D contains the simulation evidence behind Section 5, extended to all five curated rates. Appendix E examines the sensitivity of the Section 6 pairwise comparisons to correlation among the tract estimates. All numbers in the appendices are computed when this document is rendered, from data bundled with the package.

Appendix A. The variable catalog and margin-of-error rules

The default catalog covers 14 ACS variables. Table A.1 lists them together with the estimand family that governs their aggregation and margin-of-error treatment; the families implement the equations of Section 2. Variables in the `spatial_total` family are aggregated by the coverage-weighted sum of Equation (2) with the margin of error of Equation (3). The two proxy families are aggregated by the area-share weighted mean of Equation (4); the labels `median_proxy` and `area_weighted_scalar_proxy` mark that a median and a per capita mean cannot be aggregated exactly from tract values, so the reported quantity is an approximation and is flagged as such in every affected row.

Table A.1. The default variable catalog and the estimand family attached to each entry, read from the installed package.

Variable	Description	Family
B01003_001	Total population	<code>spatial_total</code>
B11001_001	Households	<code>spatial_total</code>
B17001_001	Population for whom poverty status is determined	<code>spatial_total</code>
B17001_002	Population below the poverty line	<code>spatial_total</code>
B19013_001	Median household income (dollars)	<code>median_proxy</code>
B19056_001	Households (SSI universe)	<code>spatial_total</code>
B19056_002	Households with Supplemental Security Income	<code>spatial_total</code>
B19301_001	Per capita income (dollars)	<code>area_weighted_scalar_proxy</code>
B22003_001	Households (SNAP universe)	<code>spatial_total</code>
B22003_002	Households receiving SNAP	<code>spatial_total</code>
B23025_001	Population 16 years and over	<code>spatial_total</code>
B23025_002	In labor force	<code>spatial_total</code>
B23025_003	Civilian labor force	<code>spatial_total</code>
B23025_005	Unemployed (civilian)	<code>spatial_total</code>

The five curated rates are formed from aggregation carriers drawn from this catalog. Table A.2 lists each rate with its numerator and denominator, read from the `cacs_acs_default_rates` object shipped with the package. In every case the numerator is a subset of the denominator, so the subset-proportion form of Equation (5) is applicable in principle; the package nevertheless applies the general ratio form by default, for the reasons measured in Section 5 and Appendix D. Note that the SNAP and Supplemental Security Income rates are shares of households, while the poverty rate is a share of persons and the two labor-force rates are shares of the population 16 years and over, of which the unemployment rate uses the civilian subset.

Table A.2. Aggregation carriers of the five curated rates, read from the installed package.

Rate	Numerator	Denominator
poverty_rate	B17001_002	B17001_001
snap_rate	B22003_002	B22003_001
ssi_rate	B19056_002	B19056_001
unemp_rate	B23025_005	B23025_003
labor_force_participation	B23025_002	B23025_001

When the subset-proportion form is requested and its radicand is negative for some row, the package substitutes the ratio form for that row and records the substitution in the `moe_fallback` and `moe_fallback_reason` columns. Radicand failure is not hypothetical: published tract margins occasionally violate the subset ordering, with the margin of the part exceeding the margin of the whole, and Appendix D reports how often this occurs among the Birmingham tracts.

Appendix B. The output contract

Every run returns a tibble in a canonical long schema of 27 columns: 23 mandatory columns and 4 carrier columns joined by the propagation stage. Table B.1 documents them in the order in which they appear. The design intent is that a profile can be audited from its own rows: each row names the provider and OpenStreetMap snapshot behind its isochrone, the ACS year behind its estimates, the weighting method and estimand family behind its aggregation, the margin-of-error formula that was requested and the one that was actually applied, and the origin of the failure if the value could not be computed.

Table B.1. The 27-column output contract, in column order. The four carrier columns are populated by the propagation stage and feed the rate derivations.

Column	Description
site_id	Site identifier, carried from the input table
drive_time_min	Drive-time threshold in minutes
ring_topology	Whether rings are cumulative or disjoint bands
variable	ACS variable code or derived-rate name
estimate	Catchment estimate
moe	90 percent margin of error of the estimate
weight_sum	Sum of coverage weights over intersecting tracts
n_tracts	Number of tracts entering the aggregate
n_tracts_num	Tracts behind the numerator (rate rows)

n_tracts_den	Tracts behind the denominator (rate rows)
provider	Routing provider that produced the isochrone
profile	Routing profile (for instance, driving)
osm_snapshot_date	OpenStreetMap snapshot behind the isochrone
acs_year	ACS five-year endpoint
weight_method	Weighting method (area in the current release)
estimand_family	Family of Appendix A governing the aggregation
weight_basis	Basis of the weights (coverage or area share)
moe_formula_requested	Margin-of-error formula the caller requested
moe_formula_effective	Formula actually applied to this row
moe_fallback	Whether a fallback substitution occurred
moe_fallback_reason	Reason for the fallback, if any
failure_origin	Stage in which a failure originated, if any
weight_uncertainty_propagated	Whether weight uncertainty is reflected (FALSE by design)
est_total	Carrier: coverage-weighted total
var_total_raw	Carrier: raw variance of the total
est_mean	Carrier: area-share weighted mean
var_mean_raw	Carrier: raw variance of the mean

Run-level provenance travels in attributes instead of columns, and `cacs_describe()` renders it. The following output describes the batch run of Section 6; `cacs_describe()` writes to the message stream, so the output shown here is captured from that stream.

```
cacs_describe(profiles)
```

```
#>
#> -- catchmentACS provenance map -----
#> Object type: cacs_run_result
#>
#> -- Object --
#>
#> Type: cacs_run_result
#> Rows: 1140
#> Columns: 27
#> Schema version: 1.0
#>
#> -- Run --
#>
#> Provider/profile: osrm / car
```

```

#> Drive times: 5, 10, 15,
#> Sites: 20 success / 0 failed / 20 total
#> Wall clock seconds: 0.898424
#>
#> -- Rates --
#>
#> Rate rows: 300
#> Rates computed: poverty_rate, snap_rate, ssi_rate, unemp_rate,
#> labor_force_participation,
#> Formula dispatch: general_ratio_conservative
#> Carrier-missing rate rows: 52
#> Out-of-range audit rows: 0
#> Formula downgrades: 0
#>
#> -- Cache --
#>
#> Enabled: TRUE
#> Namespace mode: production
#> Fingerprint algorithm: sha256
#> Session hits: 6
#> Session misses: 0

```

Appendix C. The batch workload

The twenty-site fixture exists so that a realistic multi-site workload can run offline, and so that the package has a frozen regression baseline. Figure C.1 maps it. The site coordinates lie on a generated grid and correspond to no real facility; the isochrones are synthetic cumulative rings that no routing engine produced; and the 911-tract panel carries placeholder identifiers in place of Census geographies, with about 5 percent of tract estimates missing to mimic the suppressed small-area values of production ACS data. The workload nevertheless exercises the same code paths as a live run, including the missing-data propagation, the per-rate carrier machinery, and the fallback bookkeeping examined in Section 6.

For completeness, the agreement check cited in Section 5 is the following comparison of the Birmingham extensive inputs against `st_interpolate_aw()`; it reproduces Table 3.

```

library(tidyr)
est <- vw$acs_sf |> st_drop_geometry() |>
  select(GEOID, variable, estimate) |>
  pivot_wider(names_from = variable, values_from = estimate)
acs_wide <- vw$acs_sf |> filter(variable == "B01003_001") |>
  select(GEOID) |> left_join(est, by = "GEOID") |>

```


(a) The twenty-site workload

(b) Cumulative rings at one site

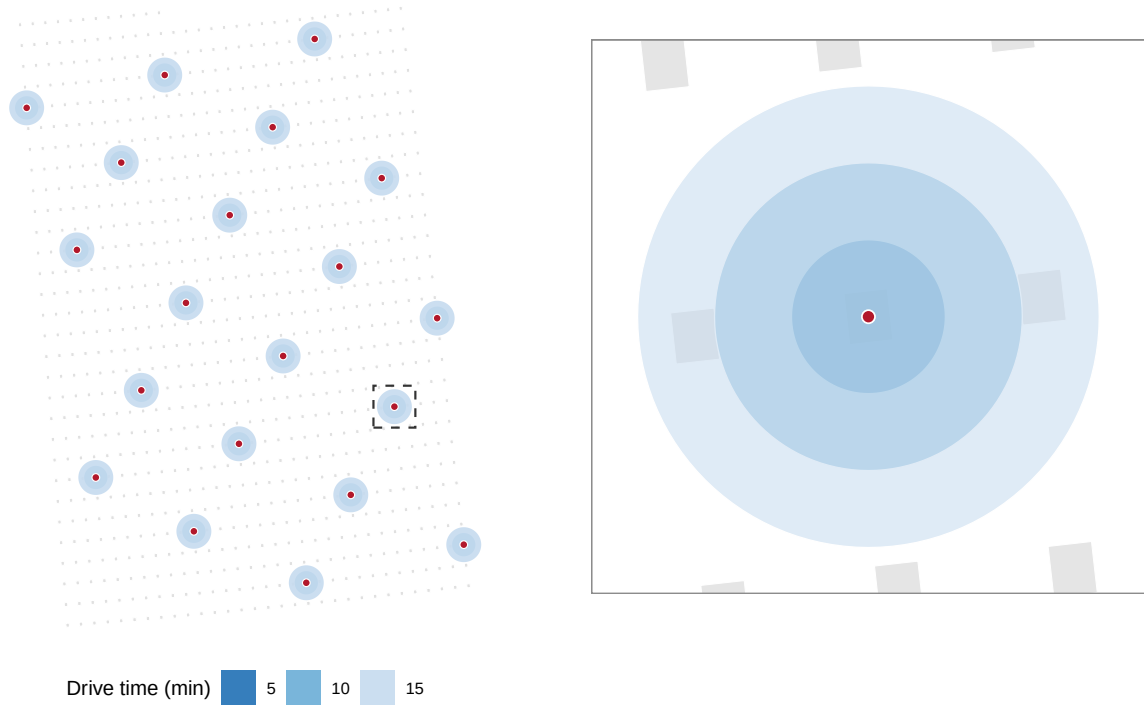


Figure C.1. The twenty-site batch workload. Panel (a) shows the 20 synthetic sites, their cumulative 5-, 10-, and 15-minute rings, and the 911-tract synthetic panel beneath them; the dashed rectangle marks the site enlarged in panel (b). No two catchments intersect at any threshold, which is the condition under which the comparison rule of Section 6 applies.

```
st_transform(5070) |> st_make_valid()
iso_eq <- st_transform(vw$iso_sf, 5070) |> st_make_valid()
aw <- st_interpolate_aw(acs_wide[c("B01003_001", "B17001_001",
                                   "B17001_002")],
                       iso_eq, extensive = TRUE)
names(aw)

#> [1] "B01003_001" "B17001_001" "B17001_002" "geometry"
```

The printed names are the whole of the interpolated layer's schema: the estimate columns and the geometry, with no uncertainty columns. This is the observation that motivated the package.

Appendix D. Simulation evidence for the margin-of-error forms

This appendix contains the simulation behind Section 5, extended from the poverty rate to all five curated rates. The design is as described there: for each rate, the numerator and the complement of the numerator within the denominator are drawn independently across tracts

from normal distributions whose standard deviations are the published margins divided by $z = 1.645$, the weighted aggregation of Section 2 is recomputed for each of 20,000 replicates, and the two published margin-of-error forms are compared with the spread of the resulting sampling distribution. The code below is the code that was executed to produce Table D.1 and Figure D.1; it also records, for each rate, the number of tracts whose published margins violate the subset ordering, in which case the complement's variance is truncated at zero.

*# Monte Carlo check of the two published margin-of-error forms for all five
curated rates in the Birmingham catchment, under the independence assumption
of Section 2. Displayed and discussed in Section 5 and Appendix D.*

```
carriers <- catchmentACS::cacs_acs_default_rates
tract_wide <- vw$acs_sf |> st_drop_geometry() |>
  select(GEOID, variable, estimate, moe)
mc_rates <- imap_dfr(carriers, function(cr, nm) {
  d <- tract_wide |> filter(variable %in% cr) |>
    pivot_wider(names_from = variable, values_from = c(estimate, moe)) |>
    inner_join(audit |> select(GEOID, area_wt), by = "GEOID")
  num_e <- d[[paste0("estimate_", cr[["num"]])]]
  den_e <- d[[paste0("estimate_", cr[["den"]])]]
  keep <- !is.na(num_e) & !is.na(den_e)
  d <- d[keep, ]; num_e <- num_e[keep]; den_e <- den_e[keep]
  sN <- d[[paste0("moe_", cr[["num"]])]] / z
  sD <- d[[paste0("moe_", cr[["den"]])]] / z
  sC <- sqrt(pmax(sD^2 - sN^2, 0))
  w <- d$area_wt
  p <- sum(w * num_e) / sum(w * den_e)
  M_num <- z * sqrt(sum(w^2 * sN^2))
  M_den <- z * sqrt(sum(w^2 * sD^2))
  m_subset <- sqrt(pmax(M_num^2 - p^2 * M_den^2, 0)) / sum(w * den_e)
  m_ratio <- sqrt(M_num^2 + p^2 * M_den^2) / sum(w * den_e)
  set.seed(42)
  sim <- replicate(20000, {
    a_rep <- rnorm(length(num_e), num_e, sN)
    c_rep <- rnorm(length(num_e), den_e - num_e, sC)
    sum(w * a_rep) / sum(w * (a_rep + c_rep))
  })
  tibble(rate = nm, estimate = p, n_neg = sum(sD^2 < sN^2),
    moe_subset = m_subset, moe_ratio = m_ratio, moe_mc = z * sd(sim),
    cov_subset = mean(abs(sim - p) <= m_subset),
    cov_ratio = mean(abs(sim - p) <= m_ratio),
```

```

sim_pov = if (nm == "poverty_rate") list(sim) else list(NULL))
})

```

Table D.1. The two published margin-of-error forms against the simulated sampling distribution, for all five curated rates in the Birmingham catchment. Violating tracts are those whose published numerator margin exceeds the denominator margin. Coverage is the share of 20,000 replicates falling inside the nominally 90 percent interval.

Rate	Estimate	Violating tracts	Subset (pp)	Ratio (pp)	Simulation (pp)	Cov. subset	Cov. ratio
poverty_rate	0.193	1	2.22	2.43	1.96	0.939	0.961
snap_rate	0.112	1	1.21	1.32	1.18	0.907	0.935
ssi_rate	0.050	0	0.81	0.84	0.80	0.905	0.917
unemp_rate	0.042	0	0.89	0.92	0.89	0.900	0.914
labor_force_participation	0.649	3	1.90	3.43	1.40	0.975	1.000

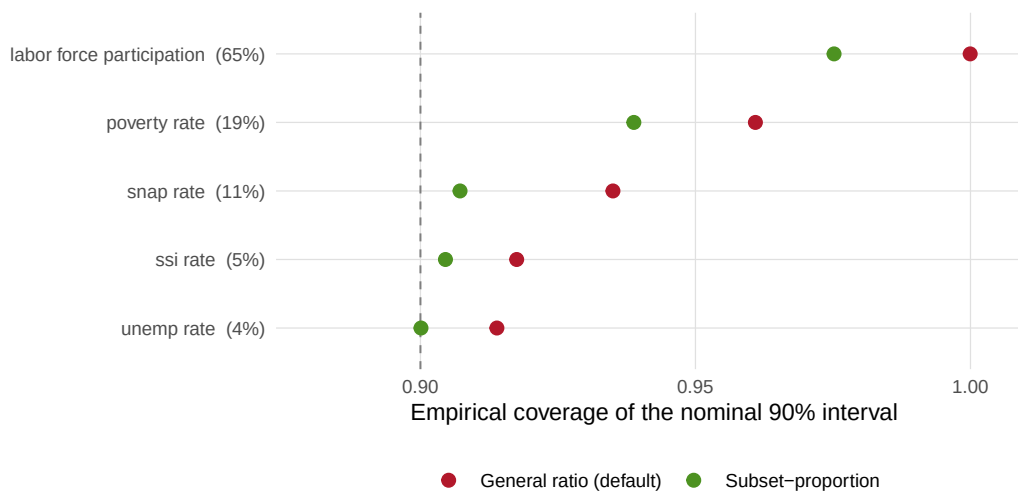


Figure D.1. Coverage of the two published margin-of-error forms against the nominal 90 percent level, for the five curated rates ordered by the size of the estimate. The conservatism of the general ratio form, which the package applies by default, grows with the level of the rate; the subset-proportion form tracks the nominal level until the rate is large.

Three observations follow from Table D.1. First, the implementation is faithful: for every rate the recomputed ratio form equals the margin the package reports, and the poverty-rate row reproduces Table 4. Second, the conservatism of the default is a function of the level of the rate, for the algebraic reason given in Section 5, so analysts working with small rates give up little by accepting the default, while analysts working with rates near one half pay the most. Third, subset-ordering violations among published tract margins are rare but present, with 5 violating tracts across the five rates in this catchment, which is the practical reason the package guards the subset-proportion form with a recorded fallback instead of assuming that the ordering holds.

Appendix E. Correlation and the comparison rule

This appendix examines what correlation among tract estimates would do to the pairwise comparisons of Section 6. For a weighted total, Equation (6) shows that a common correlation ρ inflates the variance through the cross-product term, and Table 5 reports the resulting inflation for the Birmingham population total. For a derived rate the effect is not a simple inflation, because errors that move the numerator and the denominator together partially cancel in their ratio, and the strength of the cancellation depends on how the correlation is structured across the two components. The published margins do not identify the structure, and the variance replicate tables that would identify it empirically (Folch et al., 2023) do not exist for a synthetic panel, so we bracket it with two extremes.

In structure S1, one common factor per catchment and replicate moves the poverty and non-poverty components together, which corresponds to an area-wide tendency to over- or under-count; this is the structure most favorable to cancellation. In structure S2, each component receives its own independent factor, so that poverty counts are correlated across tracts independently of the rest of the universe; this disables the cancellation. In both structures the factors are local to a catchment and independent across catchments, which matches the batch of Section 6, whose catchments lie in different counties and share no tracts. For each structure and each ρ in $\{0, 0.1, 0.2, 0.3\}$, the tract-level draws of Appendix D are given the exchangeable correlation ρ through the common factor, the catchment rates are recomputed for 4,000 replicates, each catchment's standard error is taken from its replicates, and the comparison rule of Section 6 is reapplied to every pair. The code below was executed to produce Table E.1.

```
# Sensitivity of the Section 6 pairwise count to correlation among tract  
# estimates, under two bracketing structures (Appendix E). S1 applies one  
# common factor per catchment to the poverty and non-poverty components  
# together; S2 gives each component its own factor.  
i10 <- iso |> filter(drive_time_min == 10)  
acs_pov <- acs |> st_drop_geometry() |>  
  filter(variable %in% c("B17001_001", "B17001_002")) |>  
  select(GEOID, variable, estimate, moe) |>  
  pivot_wider(names_from = variable, values_from = c(estimate, moe))  
gt <- acs |> filter(variable == "B01003_001") |> select(GEOID) |>  
  st_transform(5070) |> st_make_valid()  
gt$tract_area <- as.numeric(st_area(gt))  
inputs <- map(pov10$site_id, function(sid) {  
  ip <- i10 |> filter(site_id == sid) |> st_transform(5070) |> st_make_valid()  
  ix <- suppressWarnings(st_intersection(gt, ip))  
  ix$wt <- as.numeric(st_area(ix)) / ix$tract_area
```

```

d <- ix |> st_drop_geometry() |> select(GEOID, wt) |>
  inner_join(acs_pov, by = "GEOID") |>
  filter(!is.na(estimate_B17001_001), !is.na(estimate_B17001_002),
    wt > 1e-6)
list(w = d$wt, A = d$estimate_B17001_002, B = d$estimate_B17001_001,
  sA = d$moe_B17001_002 / z, sB = d$moe_B17001_001 / z)
})
est_chk <- map_dbl(inputs, ~ sum(.x$w * .x$A) / sum(.x$w * .x$B))
stopifnot(max(abs(est_chk - pov10$estimate)) < 1e-10)
pair_idx <- t(combn(length(inputs), 2))
share_at <- function(rho, structure, R = 4000) {
  set.seed(7)
  se <- map_dbl(inputs, function(inp) {
    k <- length(inp$A); sC <- sqrt(pmax(inp$sB^2 - inp$sA^2, 0))
    reps <- vapply(seq_len(R), function(r) {
      Za <- rnorm(1)
      Zc <- if (structure == "S1") Za else rnorm(1)
      zi <- sqrt(rho) * Za + sqrt(1 - rho) * rnorm(k)
      zj <- sqrt(rho) * Zc + sqrt(1 - rho) * rnorm(k)
      a_rep <- inp$A + inp$sA * zi
      c_rep <- (inp$B - inp$A) + sC * zj
      sum(inp$w * a_rep) / sum(inp$w * (a_rep + c_rep))
    }, numeric(1))
    sd(reps)
  })
  zst <- abs(est_chk[pair_idx[, 1]] - est_chk[pair_idx[, 2]]) /
    sqrt(se[pair_idx[, 1]]^2 + se[pair_idx[, 2]]^2)
  tibble(rho = rho, structure = structure,
    indistinct = sum(zst < z), share = mean(zst < z),
    med_se_ratio = median(se / pov10$se))
}
rho_grid <- expand_grid(rho = c(0, 0.1, 0.2, 0.3), structure = c("S1", "S2"),
  stringsAsFactors = FALSE)
rho_res <- map2_dfr(rho_grid$rho, rho_grid$structure, share_at)

```

Two features of Table E.1 deserve comment. First, at $\rho = 0$ the simulation finds 31 indistinct pairs against the 33 reported in Section 6. The difference is the conservatism measured in Appendix D: the reported margins come from the general ratio form, which is wider than the true sampling variability, so the reported analysis marks a few close pairs indistinct that the simulated standard errors can just separate. The reported count errs on

Table E.1. Sensitivity of the Section 6 pairwise count to correlation among tract estimates, under the two bracketing structures. The median SE ratio compares each catchment’s simulated standard error with the one implied by the reported general-ratio margin; the reported analysis found 33 of 153 pairs indistinct.

Structure	ρ	Indistinct pairs	Share	Median SE ratio
S1 (shared factor)	0.0	31	0.203	0.94
S1 (shared factor)	0.1	29	0.190	0.90
S1 (shared factor)	0.2	28	0.183	0.85
S1 (shared factor)	0.3	27	0.176	0.80
S2 (component factors)	0.0	30	0.196	0.95
S2 (component factors)	0.1	31	0.203	0.95
S2 (component factors)	0.2	31	0.203	0.96
S2 (component factors)	0.3	31	0.203	0.96

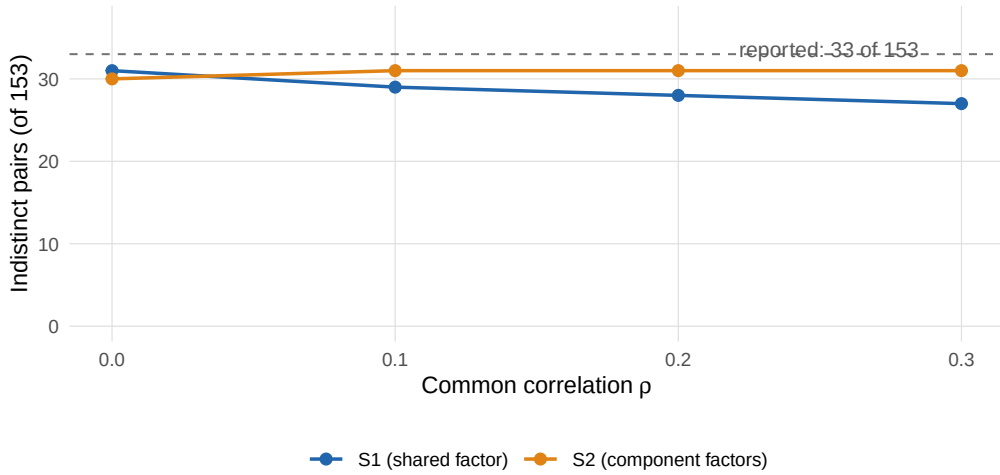


Figure E.1. The number of indistinct pairs among the 153 site comparisons of Section 6, as the correlation among tract estimates rises under the two bracketing structures. The dashed line is the count reported in Section 6 from the package’s margins. The count is insensitive to the correlation under either structure.

the cautious side, which for the funding-allocation settings that motivated the package is the appropriate direction. Second, as ρ rises the simulated standard errors shrink slightly under S1, where the cancellation operates, and grow slightly under S2, where it does not, but under neither structure does the count move by more than a few pairs across the whole range examined. The conclusion drawn in Section 6, that about one in five of these comparisons has no statistical support, does not depend on the independence assumption.